

Enterprise Connector Integration Plan

Executive Summary

Your platform already has several building blocks that are directly reusable for enterprise connector work: a multi-tenant/RBAC application model, an existing OAuth-like integration path for MCP servers, encrypted secret handling, a mature agent configuration flow, and observability/tracing surfaces. The frontend overview shows a manual REST client, route gating, reusable settings/layout patterns, and an MCP connect flow, but it also says there is no generic Connected Apps surface, no generic OAuth callback page, and no token-health or admin integration UX yet. The backend overview shows a strong FastAPI module layout, encrypted-at-rest secret handling, an OAuth start/callback pattern inside the MCP subsystem, org-scoped resources, and notifications/tracing, but it also explicitly notes that there is no generic connector layer, no webhook infrastructure, no background job queue, no token refresh scheduler, and no Redis-backed distributed control plane yet. `filecite[turn0file0]` `filecite[turn0file1]`

The right enterprise plan is not to bolt Slack, Teams, Jira, ClickUp, Gmail, Outlook, SharePoint, Google Drive, and OneDrive directly into today's MCP-specific path. The scalable approach is a **provider-neutral integration control plane**: one OAuth service, one encrypted token vault, one webhook ingress service, one queue/worker layer, one adapter contract, one audit model, and one admin policy surface. The MCP path should remain a supported integration mechanism, but enterprise connectors need a first-class integration framework because provider APIs have materially different webhook guarantees, rate limits, refresh semantics, and admin-consent models. That difference is especially large for Slack Events API, Microsoft Graph change notifications, Gmail Pub/Sub watches, and Google Drive watch channels. ¹

At a high level, I recommend a phased program. First, ship the neutral control plane and UI skeleton. Second, deliver the lowest-friction/highest-value connectors in a way that reuses token, audit, webhook, and queue primitives. Third, harden for enterprise rollout with SSO alignment, policy controls, residency/compliance settings, contract tests, chaos tests, DLQs, replay tooling, and operational dashboards. On effort and sequencing, Slack, Jira, ClickUp, Gmail, and Google Drive are good first-wave integrations; Microsoft Teams/Outlook/OneDrive/SharePoint should be implemented as a **single Microsoft Graph family** because they share identity, consent, throttling, and webhook mechanics. ²

What The Repositories Already Have

Current integration touchpoints

The frontend is integration-adjacent, but not integration-ready in a generic sense. The overview identifies a manual API layer in `lib/api/client.ts`, auth state in `contexts/auth-context.tsx`, route protection in `proxy.ts`, and an MCP-only OAuth/connect flow in `lib/api/mcp-servers.ts` plus `components/mcp/connect-mcp-sheet.tsx`. It also states that no general integration settings page, connected-apps UX, OAuth callback route, or token health indicators currently exist. That means the

existing UI has useful patterns, but not the actual control surfaces needed for enterprise connectors.

filecite turn0file0

The backend is materially ahead of the frontend in reusable primitives. The overview identifies backend/mcp_server/routes.py and backend/mcp_server/services.py as the current place where OAuth start/callback, token storage, connection tests, discovery, and runtime integration already happen. It also identifies backend/core/encryption.py for secret protection, backend/core/ratelimit.py for rate-limiting, org-scoped collections throughout the data model, notifications, and tracing. This is important: your team has already solved the shape of integration work once, but it is concentrated in the MCP subsystem rather than generalized into a provider-neutral integration layer.

filecite turn0file1

The practical inventory is therefore:

Touchpoint	Frontend state	Backend state	What it means
Auth flows	Login/auth context and dashboard route gating exist; MCP connect UI already starts OAuth in one path. filecite turn0file0	OAuth start/callback already exists in the MCP module. filecite turn0file1	Reuse the interaction pattern, but move it into a generic integration OAuth service.
Webhook handlers	No generic connector-webhook UI or callback surfaces. filecite turn0file0	Backend overview explicitly says webhook infrastructure is missing outside MCP-style flows. filecite turn0file1	Needs a dedicated ingress layer, signatures, replay protection, queueing, DLQ.
API clients	Manual REST client and MCP API client already exist. filecite turn0file0	No provider-neutral adapter layer yet. filecite turn0file1	Introduce adapter SDK boundaries by provider.
Config screens	Admin/client settings layouts exist, but no Connected Apps/Integrations module. filecite turn0file0	No backend endpoints for generic integrations yet. filecite turn0file1	Build org-level and workspace-level configuration surfaces.
Token storage	No generic UI for token state or expiry. filecite turn0file0	Encrypted token patterns already exist in the MCP path; backend recommends a new integration_tokens collection. filecite turn0file1	Reuse encryption patterns, but isolate by provider and tenant.

Touchpoint	Frontend state	Backend state	What it means
Worker hooks	No frontend token-health/refresh status. filecite0turn0file0	No job queue or refresh scheduler today; backend explicitly recommends Redis + job workers. filecite0turn0file1	A worker plane is mandatory for enterprise rollout.

What is missing right now

The most important missing pieces are not provider-specific SDK calls. They are control-plane concerns: a generic integration domain model, connector mappings to organization policy, durable event ingestion, distributed rate limiting, token lifecycle management, replay-safe webhook processing, operator visibility, and admin approval controls. The backend overview explicitly calls out the absence of webhook infrastructure, job queues, token refresh scheduling, and distributed state management; it also notes that `docker -compose` does not include Redis and that the current rate-limiting model is per-process. The frontend overview separately calls out the absence of Connected Apps, callback pages, and token-state UX. Those are the gaps that will determine whether the platform stays “integration-capable” or becomes “enterprise-integration-ready.” filecite0turn0file0 filecite0turn0file1

Enterprise Architecture For Connector Scale

Recommended control plane

The connector platform should be organized around a single internal abstraction:

Connection → Credential set → Subscription/watch → Target mapping → Event ingestion → Worker execution → Agent/tool handoff → Audit trail.

That abstraction lets you handle Slack, Microsoft Graph, Atlassian, ClickUp, Gmail, and Google Drive with the same internal lifecycle even though their external APIs differ. OAuth 2.0 authorization code flow should be the default baseline, with `state` required for callback integrity, PKCE supported by default for browser/public-client-safe flows, refresh-token handling abstracted behind the token vault, and revocation handled explicitly during disconnect or policy changes. OAuth itself is designed around limited delegated access, PKCE mitigates code interception, and token revocation exists specifically so clients can invalidate no-longer-needed grants. 3

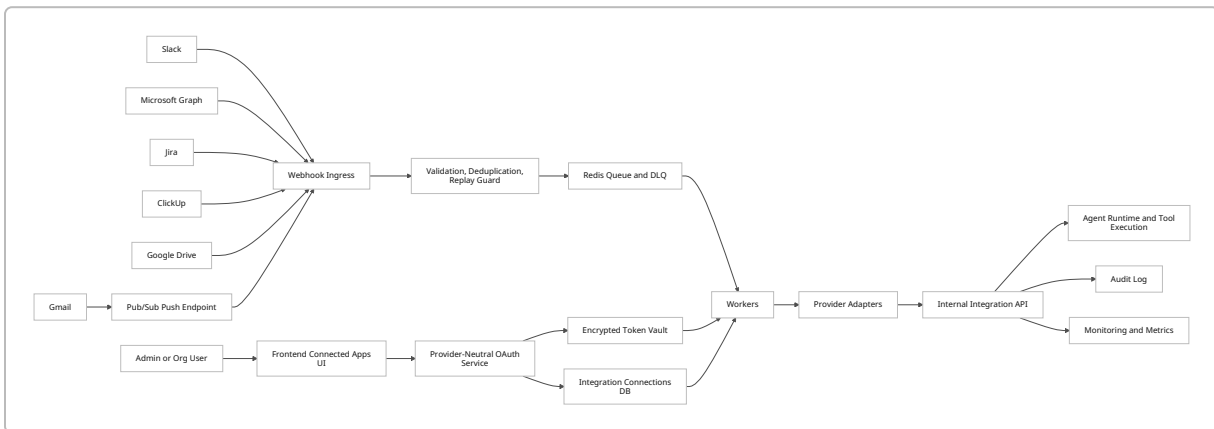
The enterprise feature set you asked for maps cleanly into this architecture:

Capability	Recommended design
Provider-neutral OAuth	One <code>IntegrationOAuthService</code> with provider metadata, auth URL generation, callback handling, PKCE/state support, incremental-consent support, and one disconnect/revoke path.

Capability	Recommended design
Encrypted token vault	Envelope encryption with a KMS-backed data-encryption-key model; store access token, refresh token, expiry, granted scopes/permissions, credential version, and revocation status separately from the visible connection record.
Token refresh / rotation	A worker-driven refresh scheduler that knows provider-specific rules: Slack rotation, Atlassian <code>offline_access</code> , Google refresh flows, Microsoft Graph refresh via Entra ID, and providers without robust refresh semantics such as ClickUp's currently non-expiring token model. ⁴
Webhook ingress / validation / retry	A dedicated ingress edge that only validates, authenticates, deduplicates, persists, and enqueues; never performs heavy business logic inline. Slack, Microsoft Graph, Gmail Pub/Sub, Google Drive, Jira, and ClickUp all reward fast acknowledgements. ⁵
Redis / job queue	Required for rate-limit coordination, idempotency windows, webhook dedupe, replay guards, refresh scheduling, DLQ, and horizontally scaled workers. The backend overview already points to Redis-backed workers as the right direction. <code>filecite turn0file1</code>
Scalable provider adapters	One base adapter interface with pluggable provider modules; keep normalization logic out of API routes.
Audit logs	Immutable integration audit records for connect/disconnect, consent change, scope escalation, webhook validation failure, replay, token refresh, admin override, and agent-triggered side effects.
RBAC / admin controls	Org-level and provider-level policies: who can connect, who can approve, whether personal vs workspace installs are allowed, whether write scopes are allowed, what channels/projects/folders are approved targets.
Rate-limit / backoff	Move all provider request execution through a policy-aware client with per-provider and per-tenant concurrency controls, token buckets, and Retry-After aware backoff.
Multi-tenant isolation	Hard partition connections, credentials, events, subscriptions, and materialized sync state by <code>organization_id</code> ; never share caches across tenants without explicit partition keys.
Monitoring / metrics	Queue depth, webhook ack latency, validation failures, refresh success, connector error rates, per-provider 401/403/409/429/5xx, DLQ volume, and policy violations.
SSO / SAML / OIDC	Keep workforce authentication separate from external-provider consent, but unify admin policy, audit, and session trust. Workforce SSO proves <i>who the operator is</i> ; connector OAuth proves <i>what external data that operator approved</i> .
Data residency / compliance	Region-pinned vaults and storage classes, provider-specific data minimization, configurable retention per connector, export/deletion workflows, and documented cross-border data processing controls.

Capability	Recommended design
Security controls	Least-privilege scopes, secret rotation, webhook signature validation, KMS-backed encryption, per-tenant RBAC, break-glass logs, egress allowlists, and redaction of connector data in logs/traces.

Architecture diagram



Key architectural choices

The most important operational choice is to **separate acknowledgement from processing**. Slack requires a 2xx response within three seconds and retries failed deliveries nearly immediately, then at roughly one minute and five minutes. Microsoft Graph considers a notification delivered if you return 2xx within three seconds, recommends `202 Accepted` if you have queued it, and keeps retrying for up to four hours with exponential backoff; it also imposes “slow” and “drop” states on endpoints that lag. Gmail push notifications are delivered through Cloud Pub/Sub, and if the push callback does not acknowledge successfully, Pub/Sub retries later. ClickUp marks a webhook as failing if the endpoint returns an unsuccessful status or takes more than seven seconds, retries up to five times per event, and can suspend the webhook based on error patterns. These guarantees make queue-first ingestion non-optional. ⁶

The second key choice is to **treat Microsoft as one connector family, not four separate engineering efforts**. Teams, Outlook, OneDrive, and SharePoint all ride on Microsoft Graph permissions, Graph throttling, and Graph change notifications. Teams does introduce a separate consideration: if you want true conversational bot experiences rather than only channel enumeration and message posting, Microsoft’s own documentation points you toward Teams bots/Bot Framework or Teams SDK patterns in addition to Graph. That should be modeled as a capability branch of the same Microsoft family, not as a totally separate tenant-auth stack. ⁷

Provider Requirements And Constraints

Connector comparison

Connector	Recommended minimum auth	Change ingestion model	Official rate-limit or quota notes	Retry / acknowledgement semantics
Slack	<code>chat:write</code> for posting; <code>channels:read</code> / <code>groups:read</code> / <code>im:read</code> / <code>mpim:read</code> for discovery; matching <code>*:history</code> scopes for readback/history. <code>chat.postMessage</code> requires <code>chat:write</code> ; <code>conversations.list</code> uses read scopes; <code>conversations.history</code> uses history scopes.	Events API subscriptions for the subscribed events your product actually needs; queue inbound deliveries immediately.	Method tiers vary by API: Tier 2 is 20+ per minute, Tier 3 is 50+ per minute, Tier 4 is 100+ per minute. <code>chat.postMessage</code> is effectively special-tier and generally allows about 1 message/second/channel. Events API allows 30,000 deliveries per workspace/app per 60 minutes.	Return HTTP 2xx within 3 seconds. Slack retries failed deliveries up to 3 times, nearly immediately, then after about 1 minute and 5 minutes, and includes <code>x-slack-retry-num</code>.
Microsoft Teams	<code>Team.ReadBasic.All</code> to list joined teams, <code>Channel.ReadBasic.All</code> to list channels, <code>ChannelMessage.Send</code> to post channel/chat messages. If you want bot-style conversations, plan a Teams bot/Bot Framework capability in addition to Graph.	Microsoft Graph change notifications for Teams resources; conversational bot traffic uses Teams/Bot Framework activity delivery.	Microsoft Graph throttling varies by scenario and request type. 429 responses carry <code>Retry-After</code>; Microsoft explicitly recommends using change tracking/notifications rather than polling. Teams change-notification subscriptions share a tenant cap of 10,000 across Teams resources.	Webhook endpoints must respond within 3 seconds; Microsoft recommends <code>200</code> if processed and <code>202</code> if queued. Notifications are retried with exponential backoff for up to 4 hours; subscription setup requires validation-token handling.

Connector	Recommended minimum auth	Change ingestion model	Official rate-limit or quota notes	Retry / acknowledgement semantics	E in c
Jira Cloud	<code>offline_access</code> for refresh tokens; <code>read:jira-user</code> , <code>read:jira-work</code> , <code>write:jira-work</code> for baseline issue/user integration.	Webhooks for issue and comment lifecycle. For the common issue path, start with <code>jira:issue_created</code> , <code>jira:issue_updated</code> , <code>jira:issue_deleted</code> ; add comment events only if needed.	Jira rate limiting returns 429 plus <code>Retry-After</code> ; rate-limit headers also include <code>X-RateLimit-Reset</code> and reasons such as burst or quota exhaustion.	Respect <code>Retry-After</code> ; Atlassian recommends exponential backoff with jitter. For OAuth apps, dynamic webhook registration is limited to 5 webhooks per app per user on a tenant. In the docs reviewed, webhook-registration/security details are explicit; delivery scheduling should still be treated as at-least-once and idempotent on your side.	M C a w d s G b n n
ClickUp	OAuth authorization-code flow; the official auth page reviewed documents workspace authorization rather than a granular scope model. Tokens currently do not expire, though ClickUp says that is subject to change.	Webhooks by workspace/location. Common task events include <code>taskCreated</code> , <code>taskUpdated</code> , <code>taskDeleted</code> , plus more granular task/list/folder/space events.	Rate limits are per token and depend on workspace plan: 100 rpm on lower plans, 1,000 rpm on Business Plus, and 10,000 rpm on Enterprise. 429 responses include <code>X-RateLimit-Limit</code> , <code>X-RateLimit-Remaining</code> , and <code>X-RateLimit-Reset</code> .	Requests are signed with HMAC SHA-256 using a webhook secret in <code>X-Signature</code> . Webhooks are considered failing on unsuccessful status or >7-second responses; ClickUp retries up to 5 times per event, can suspend on repeated failures, and treats 401/410 specially.	M S th G b d b a s e

Connector	Recommended minimum auth	Change ingestion model	Official rate-limit or quota notes	Retry / acknowledgement semantics	E in c
Gmail	<code>gmail.readonly</code> or <code>gmail.metadata</code> / <code>gmail.readonly</code> for ingestion; <code>gmail.send</code> or <code>gmail.compose</code> for outbound send; <code>gmail.modify</code> only if your workflow changes labels/threads.	Gmail <code>watch</code> to Cloud Pub/Sub; notifications contain mailbox identity plus <code>historyId</code> , and your backend must call <code>history.list</code> to resolve actual changes.	Gmail quotas are unit-based: 1,200,000 quota units per minute per project and 6,000 per minute per user per project. Gmail also enforces per-user sending and concurrent-request limits. Push notifications have a maximum rate of 1 event/second per watched user; excess notifications are dropped.	Re-call <code>watch</code> at least every 7 days; Google recommends once per day. Webhook push delivery is acknowledged by a successful response, and Pub/Sub retries later if not acknowledged. API retries should use exponential backoff starting at least 1 second after failure.	M b S r c d n r n
Outlook	<code>Mail.ReadBasic</code> or <code>Mail.Read</code> for inbox access, <code>Mail.Send</code> for outbound mail.	Microsoft Graph change notifications on message resources such as <code>/me/messages</code> or specific mail-folder message collections.	Same Microsoft Graph throttling model: 429 with <code>Retry-After</code> , variable limits by scenario, and explicit guidance to prefer change tracking/notifications over polling.	Same Graph webhook model: 2xx within 3 seconds, queue-heavy work, validation token handling, exponential retries up to 4 hours.	M th n p M G
SharePoint	<code>Sites.Read.All</code> for site discovery; <code>Files.Read</code> / <code>Files.Read.All</code> or <code>Sites.Read.All</code> for document libraries depending depth and breadth.	Microsoft Graph change notifications on site/list/drive resources, plus Graph drive APIs for library traversal.	Same Microsoft Graph throttling model.	Same Graph webhook model with validation token and queue-first processing.	H e t r c s d li g

Connector	Recommended minimum auth	Change ingestion model	Official rate-limit or quota notes	Retry / acknowledgement semantics	E in c
Google Drive	Prefer <code>drive.file</code> with Picker for narrow, user-selected access; use <code>drive.readonly</code> or broader scopes only when product requirements truly need full-drive traversal.	<code>files.watch</code> for per-file notifications or <code>changes.watch</code> for broader change streams; normalize <code>sync</code> , <code>add</code> , <code>remove</code> , <code>update</code> , <code>trash</code> , <code>untrash</code> , and <code>change</code> .	Drive quotas are unit-based: 1,000,000 quota units/min/project and 325,000/min/user/project; <code>files.list</code> costs more units than <code>files.get</code> , and 403/429 should be handled with truncated exponential backoff.	Watch channels carry <code>X-Goog-*</code> headers, optional channel tokens, and explicit expiration. There is no automatic renewal ; files-watch channels cap at 1 day and changes-watch at 1 week. Replace channels before expiry.	M b w q b a g in
OneDrive	<code>Files.Read</code> for least-privilege delegated access, <code>Files.Read.All</code> for broader app-wide access; <code>Sites.Read.All</code> when traversing SharePoint-backed drives.	Microsoft Graph drive and <code>driveItem</code> change notifications.	Same Microsoft Graph throttling model.	Same Graph webhook model with validation token and queue-first processing.	M tl tl G C a fi c c s

Practical provider notes

Slack, Jira, ClickUp, Gmail, and Google Drive can all be delivered as “business app connectors” without introducing a second product mode. Teams is the outlier. If the product requirement is “post to a channel, list teams/channels, attach references to chats,” Graph is enough. If the requirement is “the platform behaves like a conversational assistant inside Teams chats/channels,” Microsoft’s own documentation points toward Teams bot capabilities or Teams SDK/Bot Framework patterns on top of Graph. I would therefore split Teams into **Graph channel integration** and **Teams conversational bot mode**, and not let the latter block the first. ¹⁷

Google’s connectors deserve special operational treatment. Gmail and Drive are not conventional “webhook POST body contains the changed record” systems in the way ClickUp or many custom SaaS webhooks are. Gmail sends a mailbox watch notification that carries a `historyId`; the application has to query `history.list` to resolve actual mail changes. Google Drive notifications are header-driven watch-channel signals with explicit expiration and no auto-renewal. That means your integration framework should have a first-class concept of **subscription/watch renewal** and **cursor-based reconciliation**, not only webhook receiving. ¹⁸

Target Backend And Frontend Design

Proposed backend module structure

The current MCP module proves the team can solve OAuth/token/runtime problems, but it is too provider-specific to be the permanent home of enterprise connectors. I recommend the following structure:

Proposed module	Responsibility	Why it exists
<code>backend/integrations/registry.py</code>	Provider catalog, capability metadata, scope manifests, consent templates	Keeps connector definitions declarative and reviewable
<code>backend/integrations/oauth/service.py</code>	Start OAuth, callback handling, PKCE/state, token exchange, revoke/disconnect	One neutral OAuth path across all providers
<code>backend/integrations/vault/service.py</code>	Encrypt/decrypt tokens, rotate credential versions, expose safe metadata only	Separates secrets from normal connection records
<code>backend/integrations/adapters/base.py</code>	Base adapter contract	Stable interface for all providers
<code>backend/integrations/adapters/slack.py</code>	Slack API + Events normalization	Provider-specific logic stays isolated
<code>backend/integrations/adapters/microsoft.py</code>	Graph client for Teams/Outlook/OneDrive/SharePoint	Treat Microsoft as one auth/throttle family
<code>backend/integrations/adapters/jira.py</code>	Jira REST + webhook registration	Jira-specific event/project mapping
<code>backend/integrations/adapters/clickup.py</code>	ClickUp API + webhook registration	Task/workspace mapping and signature checks
<code>backend/integrations/adapters/google_mail.py</code>	Gmail watch/history sync	Pub/Sub watch lifecycle and <code>historyId</code> handling
<code>backend/integrations/adapters/google_drive.py</code>	Drive watch/channel lifecycle	Expiration, channel replacement, change normalization
<code>backend/integrations/webhooks/ingress.py</code>	Public receiver endpoints	Fast acknowledgement, auth, dedupe, enqueue
<code>backend/integrations/webhooks/validate.py</code>	Signature/token validation per provider	Centralizes security logic

Proposed module	Responsibility	Why it exists
<code>backend/integrations/workers/*.py</code>	Refresh jobs, webhook processing, subscription renewal, replay, DLQ handling	Moves slow work off request path
<code>backend/integrations/policies/service.py</code>	Org-level allow/deny, write-scope restrictions, approved targets	Enterprise RBAC/admin control layer
<code>backend/integrations/audit/service.py</code>	Immutable audit/event logs	Compliance and incident response
<code>backend/integrations/api/routes.py</code>	Generic connection/subscription/health/admin endpoints	Stable frontend contract

This design lines up naturally with the backend's current strengths: modular route/service organization, existing encryption patterns, org scoping, notifications, and tracing. It also addresses the backend overview's explicit recommendation to add `backend/integrations/`, a token store, Redis-backed workers, and dedicated webhook handlers. `filecite turn0file1`

Proposed database schema

The core database design should separate **connection identity**, **credential material**, **subscriptions**, **queueable events**, **sync cursors**, and **audit**. That separation is what allows safe admin UX and clean operational controls.

Core tables

Table	Key fields	Purpose
<code>integration_connections</code>	<code>id</code> , <code>organization_id</code> , <code>provider</code> , <code>provider_account_id</code> , <code>display_name</code> , <code>status</code> , <code>connected_by_user_id</code> , <code>install_mode</code> , <code>consent_type</code> , <code>granted_scope_hash</code> , <code>last_health_at</code> , <code>last_error_code</code> , <code>created_at</code> , <code>updated_at</code>	Canonical visible record for a connected app/workspace/account
<code>integration_credentials</code>	<code>id</code> , <code>connection_id</code> , <code>credential_version</code> , <code>encrypted_access_token</code> , <code>encrypted_refresh_token</code> , <code>token_type</code> , <code>expires_at</code> , <code>refresh_after</code> , <code>rotation_policy</code> , <code>revoked_at</code> , <code>created_at</code>	Secret material, versioned and vault-backed

Table	Key fields	Purpose
integration_targets	id, connection_id, target_type, provider_target_id, display_name, config_json, is_default, created_at	Connected channel/project/folder/mailbox/library mappings
integration_subscriptions	id, connection_id, provider_subscription_id, resource_path, change_type, verification_secret_ref, status, expires_at, renew_after, created_at	Webhook/watch/subscription lifecycle state
integration_events	id, organization_id, provider, connection_id, provider_event_id, provider_delivery_id, normalized_event_type, event_time, payload_ref, received_at, validated_at, dedupe_key, status, retry_count, next_attempt_at, dead_lettered_at	Durable ingress ledger and worker handoff
integration_sync_cursors	id, connection_id, resource_key, cursor_type, cursor_value, checkpoint_time, updated_at	Gmail historyId, Google Drive change page state, Graph delta tokens, Slack cursors if needed
integration_audit_logs	id, organization_id, actor_user_id, connection_id, action, resource_type, resource_id, before_json, after_json, ip, user_agent, created_at	Compliance and forensics

Table	Key fields	Purpose
integration_policy_overrides	id, organization_id, provider, allow_connect, allow_write_scopes, approved_scope_set, admin_approval_required, allowed_install_modes, data_residency_region, retention_days, updated_by_user_id, updated_at	Org-level governance
integration_delivery_attempts	id, event_id, attempt_number, started_at, finished_at, outcome, http_status, error_code, worker_id	Fine-grained replay / DLQ / SRE debugging

Proposed API endpoints

Method	Endpoint	Purpose
GET	/integrations/providers	Provider catalog and capabilities
GET	/integrations/connections	List org connections
POST	/integrations/connections/{provider}/start-oauth	Generate auth URL + state/PKCE
POST	/integrations/connections/{provider}/complete-oauth	Exchange code and create connection
POST	/integrations/connections/{id}/disconnect	Revoke/disconnect connection
GET	/integrations/connections/{id}	Connection details and target mappings
PATCH	/integrations/connections/{id}	Rename/update connection metadata
GET	/integrations/connections/{id}/health	Token/subscription health view
POST	/integrations/connections/{id}/refresh	Force token refresh or subscription renewal
GET	/integrations/connections/{id}/targets	List workspaces/channels/projects/folders
POST	/integrations/connections/{id}/targets	Save target mappings

Method	Endpoint	Purpose
GET	/integrations/connections/{id}/subscriptions	List provider subscriptions/watch channels
POST	/integrations/connections/{id}/subscriptions	Create/renew subscriptions
DELETE	/integrations/subscriptions/{id}	Remove subscription
POST	/integrations/webhooks/slack	Slack ingress
POST	/integrations/webhooks/microsoft	Graph ingress
POST	/integrations/webhooks/jira	Jira ingress
POST	/integrations/webhooks/clickup	ClickUp ingress
POST	/integrations/webhooks/google/pubsub	Gmail/Drive Pub/Sub push ingress
GET	/integrations/audit	Search integration audit logs
GET	/integrations/admin/policies	Provider policies for the org
PATCH	/integrations/admin/policies/{provider}	Update policy / admin controls

Proposed frontend UX flows

The frontend overview already suggests where this should live: a dedicated integrations area under dashboard settings or as a first-class dashboard section, plus provider-specific connect flows and a callback handler. The overview also lists the absence of those pages today and even sketches likely routes like `/client/integrations/callback`. That makes the UI plan high-confidence rather than speculative.

filecite turn0file0

The UX should be organized around four screens:

Screen	Primary user	Role
Connected Apps index	Org admins and permitted power users	See connection inventory, health, scopes, targets, errors
OAuth callback / completion	Any user initiating connection	Complete handshake, display success/error, reconcile pending state
Connection detail / token health	Org admins	Review scopes, expiry, targets, subscriptions, recent events
Admin policy config	Org owners / security admins	Restrict providers, write scopes, target classes, data handling, staff permissions

Suggested wireframes

Connected Apps

```

+
| Connected
Apps
| Search [_____] Filter: Provider [All] Status [All] + Connect
App |
+-----+
+
| Slack Connected Healthy 3 targets Last sync 2m ago
[Manage] |
| Jira Connected Warning Token refresh due today
[Manage] |
| Gmail Connected Healthy Inbox watch active
[Manage] |
| Teams Not connected
[Connect] |
| Google Drive Connected Error Channel renewal failed
[Manage] |
+-----+
+
| Bulk actions: Export audit | View failed events | Policy
center |
+-----+
+

```

OAuth callback

```
+-----+
| Connecting Microsoft Teams |
| Verifying state...        |
| Exchanging authorization code... |
| Reading tenant and granted permissions... |
|                             |
| [Success] Connection created |
| Tenant: Contoso            |
| Permissions: Team.ReadBasic.All, Channel.ReadBasic.All... |
|                             |
| [Choose targets]   [Back to Connected Apps] |
+-----+
```

Connection detail / health

```

+-----+
+
| Connection: Gmail /
support@company.com
| Status: Healthy      Last refresh: 09:14 UTC      Watch renewal: due in 18
hrs
+-----+
+
| Granted
access
| -
gmail.readonly
|
| -
gmail.send
|
|
|
|
Targets
|
| -
Inbox
| - Support
label
|
|
|
Events
|
| - 09:11 mailbox update
processed
| - 08:55 history sync checkpoint
advanced
| - 08:20 watch
renewed
|
|
| [Test connection] [Rotate targets] [Disconnect] [View
audit]
+-----+
+

```

Admin policy center

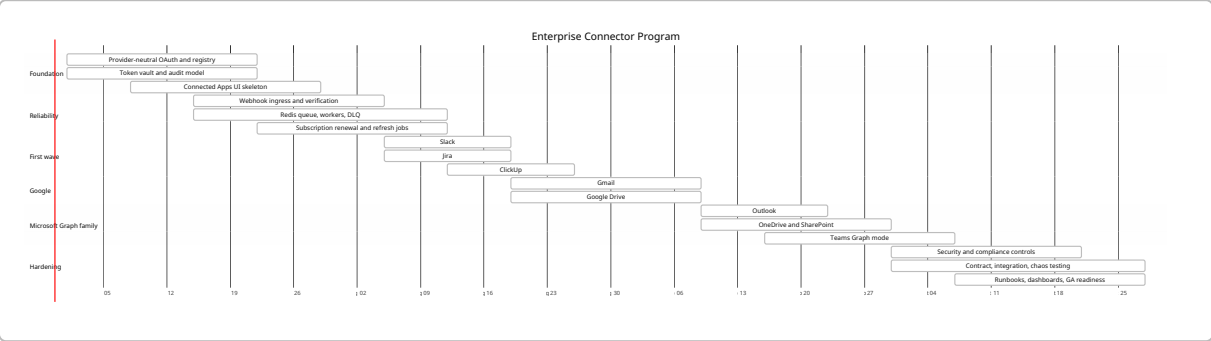
Roadmap Risk Mitigation And Assurance

Phased implementation roadmap

Phase	Scope	Priority	Effort	Why this phase goes here	Core failure-mode mitigation
Foundation	Generic integration domain, token vault, OAuth control plane, provider registry, Connected Apps UI skeleton, audit log model	Highest	High	Every provider depends on this	Vault encryption, state/PKCE, callback replay guards, admin policy gates
Ingress and workers	Webhook ingress, Redis queue, idempotency keys, event ledger, DLQ, subscription renewal scheduler, refresh workers	Highest	High	Required before real production traffic	Fast ack + queue, Retry-After aware clients, DLQ and replay tooling
First-wave connectors	Slack, Jira, ClickUp	High	Medium	Strong business value, comparatively simpler rollout than Microsoft/ Gmail	Per-provider backoff, webhook verification, target-scoped installations
Google wave	Gmail and Google Drive	High	Medium-High	Shared Google identity model; Pub/Sub/watch patterns need the worker layer first	Cursor reconciliation, watch renewal jobs, fallback sync windows
Microsoft wave	Outlook, OneDrive, SharePoint, Teams Graph mode	High	High	Shared Graph-family implementation gives best leverage	Validation-token handling, subscription renewal, admin-consent gating

Phase	Scope	Priority	Effort	Why this phase goes here	Core failure-mode mitigation
Teams conversation mode	Teams bot / notification-bot capability if required	Medium	High	Valuable, but only if the product truly needs in-Teams conversation UX	Separate bot service surface, conversation reference persistence, proactive-message throttling
Enterprise hardening	SSO/policy alignment, compliance controls, dashboards, runbooks, support tooling	Highest for GA	Medium-High	Makes the system enterprise-ready rather than merely feature-complete	Audit evidence, residency/retention controls, chaos testing

Timeline view



Explicit remediation tasks from current repositories and plan

These are the concrete missing items I would add to the implementation backlog now. They are based on the repository overviews you supplied, and they intentionally exclude the already-resolved build/config concerns you told me not to include. `filecite turn0file0` `filecite turn0file1`

Remediation task	Why it is needed now
Create <code>backend/integrations/*</code> as a first-class domain	Current reusable logic is MCP-centric, not generic.
Add Redis-backed queue and worker runtime	Backend overview explicitly calls out missing queue/distributed state and per-process rate limiting.

Remediation task	Why it is needed now
Introduce <code>integration_connections</code> , <code>integration_credentials</code> , <code>integration_subscriptions</code> , <code>integration_events</code> , and <code>integration_audit_logs</code>	Current data model does not represent enterprise connectors as first-class entities.
Build generic webhook ingress endpoints with per-provider validation	Slack, Graph, Jira, ClickUp, Gmail, and Drive all depend on reliable ingress semantics.
Add token-refresh and subscription-renewal scheduler	Required for Slack rotation, Atlassian refresh tokens, Graph subscriptions, Gmail watches, and Drive channels. ¹⁹
Build Connected Apps UI and generic callback page	Frontend overview states these do not exist yet. <code>filecite</code> <code>turn0file0</code>
Add token-health, last-sync, and last-error UX	Enterprise operators need operational visibility before support incidents arrive.
Add provider policy controls and admin approval workflow	Needed for least-privilege and enterprise governance.
Add DLQ, replay, and failed-delivery inspection tools	Necessary for webhook-heavy systems under throttling/outage conditions.
Treat Microsoft as one shared connector family	Avoid duplicated OAuth, token, and subscription logic across Teams/Outlook/OneDrive/SharePoint.
Decide whether Teams conversational bot mode is in scope for phase one	This materially changes backend/runtime architecture.

Security, compliance, and testing requirements

The security baseline should be stricter than the rest of the product because connectors extend trust into third-party systems. Slack signs requests with `X-Slack-Signature`; ClickUp signs webhooks with an HMAC in `X-Signature`; Jira supports secret-based HMAC validation in `X-Hub-Signature`; Microsoft Graph requires endpoint validation plus strict timing/availability behavior. Google Drive watch channels can include a channel token in `X-Goog-Channel-Token`, and Gmail push/webhook acknowledgement happens through Cloud Pub/Sub semantics. All of those should terminate into a common ingress layer that verifies authenticity before enqueueing. ²⁰

For compliance and platform security, I recommend the following non-negotiables:

Area	Requirement
Least privilege	Request only the minimum scopes/permissions needed per provider and capability; split read-only versus write-enabled connections.

Area	Requirement
Secrets management	KMS/HSM-backed envelope encryption, credential versioning, rotation workflows, and no secrets in application logs or client payloads.
Tenant isolation	Every connection, credential, event, cursor, and subscription must be hard-scoped to <code>organization_id</code> and checked in every worker path.
Auditability	Immutable logs for connect/disconnect, scope escalation, target changes, admin overrides, token refresh failures, and replay attempts.
Data minimization	Store metadata and cursors by default; avoid retaining full third-party content unless a product feature truly requires it.
Residency and retention	Region-selected storage for connection metadata and vault records where required; connector-specific retention periods and deletion workflows.
Admin controls	Separate who may connect, who may approve connectors, and who may permit write scopes or broad-drive/site access.
Incident response	Replay tooling, DLQ browsing, event search, revocation/disconnect workflows, and emergency provider shutdown toggles.

Testing must also go beyond ordinary CRUD tests:

Test type	What to validate
Integration tests	End-to-end OAuth, token refresh, disconnect/revoke, webhook receipt, subscription renewal, and target mapping against provider sandboxes/test tenants
Contract tests	Provider response/header contracts, signature verification, <code>Retry-After</code> parsing, Graph validation-token handling, Pub/Sub payload decoding
Penetration tests	OAuth callback CSRF/state abuse, webhook spoofing, replay attacks, tenant-boundary bypass, secret exfiltration, SSRF through integration endpoints
Load tests	Burst webhook delivery, queue saturation, per-provider 429 response storms, concurrent refresh jobs
Chaos tests	Redis outage, worker crash mid-delivery, duplicate events, out-of-order events, expired subscriptions, Pub/Sub retry waves, Graph slow/drop states
Recovery tests	DLQ replay, token revocation recovery, webhook secret rotation, provider outage fallback, cursor rebuild after missed notifications
UI tests	Connect/disconnect, error states, permission escalations, token health banners, admin approval flow

Open questions and limitations

This report is grounded in the two repository-overview documents you supplied plus official provider documentation, not a fresh statement-by-statement audit of every source file in the repositories. Repo-

derived claims therefore reflect those overview documents rather than a new direct code walkthrough of each underlying file. `filecite\turn0file0` `filecite\turn0file1`

Two strategic questions remain important. First, do you want **Teams as a messaging/channel connector** or **Teams as an in-product conversational bot surface**? The first fits naturally into the shared Microsoft Graph family; the second adds a distinct bot-service delivery model. Second, for Google Drive and SharePoint/OneDrive, do you want **narrow user-selected file access** or **broad tenant/library indexing**? The answer will materially change scope selection, admin-consent burden, residency/compliance exposure, and review time. Official docs strongly reward staying narrow where possible, especially for Google Drive's `drive.file` pattern and Microsoft's least-privilege permission guidance. ²¹

¹ ⁵ ⁶ <https://api.slack.com/apis/connections/events-api>

<https://api.slack.com/apis/connections/events-api>

² ⁷ <https://learn.microsoft.com/en-us/graph/throttling>

<https://learn.microsoft.com/en-us/graph/throttling>

³ <https://www.rfc-editor.org/rfc/rfc6749>

<https://www.rfc-editor.org/rfc/rfc6749>

⁴ ¹⁹ <https://docs.slack.dev/authentication/using-token-rotation/>

<https://docs.slack.dev/authentication/using-token-rotation/>

⁸ <https://docs.slack.dev/reference/methods/chat.postMessage/>

<https://docs.slack.dev/reference/methods/chat.postMessage/>

⁹ <https://learn.microsoft.com/en-us/graph/api/user-list-joinedteams?view=graph-rest-1.0>

<https://learn.microsoft.com/en-us/graph/api/user-list-joinedteams?view=graph-rest-1.0>

¹⁰ <https://developer.atlassian.com/cloud/jira/platform/oauth-2-3lo-apps/>

<https://developer.atlassian.com/cloud/jira/platform/oauth-2-3lo-apps/>

¹¹ <https://developer.clickup.com/docs/authentication>

<https://developer.clickup.com/docs/authentication>

¹² <https://developers.google.com/workspace/gmail/api/auth/scopes>

<https://developers.google.com/workspace/gmail/api/auth/scopes>

¹³ <https://learn.microsoft.com/en-us/graph/api/user-list-messages?view=graph-rest-1.0>

<https://learn.microsoft.com/en-us/graph/api/user-list-messages?view=graph-rest-1.0>

¹⁴ <https://learn.microsoft.com/en-us/graph/api/site-search?view=graph-rest-1.0>

<https://learn.microsoft.com/en-us/graph/api/site-search?view=graph-rest-1.0>

¹⁵ ²¹ <https://developers.google.com/workspace/drive/api/guides/api-specific-auth>

<https://developers.google.com/workspace/drive/api/guides/api-specific-auth>

¹⁶ <https://learn.microsoft.com/en-us/graph/api/drive-list?view=graph-rest-1.0>

<https://learn.microsoft.com/en-us/graph/api/drive-list?view=graph-rest-1.0>

¹⁷ <https://learn.microsoft.com/en-us/graph/api/chatmessage-post?view=graph-rest-1.0>

<https://learn.microsoft.com/en-us/graph/api/chatmessage-post?view=graph-rest-1.0>

18 <https://developers.google.com/gmail/api/guides/push>

<https://developers.google.com/gmail/api/guides/push>

20 <https://docs.slack.dev/authentication/verifying-requests-from-slack/>

<https://docs.slack.dev/authentication/verifying-requests-from-slack/>